

กลุ่ม 11 — Ownership

1. สิทธิการ ปุณกะบุตร — Concept + Short Code Illustration
2. อังกฤษ ถ้ำสุวรรณ — Detailed Code + Live Demo
3. ภูริณัฐ สุวรรณสังโส — Rust vs Other Language + PPL Analysis
4. Sothea Sokea — Exercises + Common Mistakes + Challenge

Agenda — โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

1

Member 1

**Concept + Short Code
Illustration**

แนวคิดหลักของหัวข้อ พร้อมโค้ดตัวอย่างสั้น ๆ

~5 นาที

2

Member 2

Detailed Code + Live Demo

อธิบายโค้ดเชิงลึกและสาธิตการทำงานจริง

~5 นาที

3

Member 3

**Rust vs Other Language +
PPL Analysis**

เปรียบเทียบภาษา และวิเคราะห์เชิง
Programming Language

~5 นาที

4

Member 4

**Exercises + Common
Mistakes + Challenge**

ข้อผิดพลาดที่พบบ่อย และโจทย์ท้าทายผู้ฟัง

~5 นาที

Ownership คืออะไร? (Concept)

- **Ownership** คือ ระบบ (set of rules) ที่ Rust ใช้จัดการหน่วยความจำ โดยไม่ต้องมี Garbage Collector หรือ Manual Management

มีกฎ 3 ข้อ:

- (1) ทุกๆ ค่า ใน Rust จะมี "เจ้าของ" (Owner) เสมอ
- (2) เมื่อ owner หลุด scope ค่านั้นถูก drop ทันที โดยอัตโนมัติ
- (3) มีเจ้าของได้เพียง "คนเดียว" เท่านั้นในเวลาเดียวกัน

ช่วยแก้ปัญหา

→ ความปลอดภัยของหน่วยความจำที่พบบ่อยในการเขียนโปรแกรม ได้แก่

- dangling pointer
- double free
- memory leak ที่พบบ่อยในภาษาอย่าง C/C++ ไม่ต้องเขียน free() เอง compiler จัดการคืนหน่วยความจำให้อัตโนมัติ

```
fn main() {  
    let s = String::from("hello");  
    println!("{}", s);  
} // s หลุด scope ที่นี่ -> ถูก drop อัตโนมัติ
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
hello
```

Move Semantics คืออะไร? (Concept)

- **Move** คือการ "ย้าย" ความเป็นเจ้าของ (ownership) จากตัวแปรเดิมไปยังตัวแปรใหม่
- Rust ออกแบบให้ตัวแปรเดิมใช้งานต่อไม่ได้ทันทีหลัง move เพื่อป้องกันปัญหา double free
- **แก้ปัญหา**การที่สองตัวแปรชี้ไปยังข้อมูล heap เดียวกัน แล้วพยายาม drop ซ้ำตอน scope จบ
- **ตัวแปรที่ถูก move แล้ว compiler จะบล็อกไม่ให้ใช้ต่อ (compile error)**

```
fn main() {  
    let s1 = String::from("hello");  
        let s2 = s1; // ownership ย้ายจาก s1 ไป s2  
    println!("{}", s2);  
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
hello
```

Copy & Clone คืออะไร? (Concept)

- **Copy Trait:** type พื้นฐานบน stack (i32, bool, char, f64) จะถูก copy อัตโนมัติแทนการ move เพราะคิดลอกแบบ cost น้อย
- **Clone:** สำหรับข้อมูลบน heap (เช่น String) ต้องเรียก .clone() เพื่อคัดลอกข้อมูลจริงแบบ deep copy อย่างชัดเจนแก้ปัญหากรณีที่ต้องการข้อมูลชุดใหม่จริงๆ โดยไม่กระทบตัวแปรต้นฉบับ
- **ช่วยแก้ปัญหา**กรณีต้องการใช้ตัวแปรสองตัวพร้อมกัน โดยไม่ทำให้ตัวแปรเดิมถูก move ที่ง
- **Copy ทำอัตโนมัติ (cost น้อย) ส่วน Clone ต้องเขียนเอง (มี cost ด้าน performance)**

```
fn main() {
    let x = 5;
    let y = x;           // copy อัตโนมัติ (stack)
    let s1 = String::from("hello");
    let s2 = s1.clone();  // ต้องเรียก clone เอง
                          (heap)
    println!("{}", x, y, s1, s2);
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
5 5 hello hello
```


Scope คืออะไร? (Concept)

- **Scope** คือขอบเขตของตัวแปรในโปรแกรม ตั้งแต่จุดที่ถูกประกาศจนถึงจุดที่ปิดวงเล็บ {}
- Rust ผูก ownership เข้ากับ scope โดยตรง ตัวแปรมีผลใช้งานได้เฉพาะภายใน scope ของมันเท่านั้น
- **ช่วยแก้ปัญหา**การจัดการหน่วยความจำแบบ Manual เพราะไม่ต้องกำหนดจุดคืนหน่วยความจำเอง(free)
- **เมื่อออกจาก scope** ตัวแปรที่เป็น owner จะถูก drop กันทีโดยอัตโนมัติ

```
fn main() {  
    {  
        let s = String::from("hello"); // s เริ่มมีผล  
        println!("{}", s);  
    } // s หลุด scope ที่นี่ -> ถูก drop  
    // println!("{}", s); // error: s ไม่มีผลแล้วนอก scope  
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
hello
```

Memory Management คืออะไร? (Concept)

- **Memory Management** คือการจัดการหน่วยความจำที่โปรแกรมขอใช้ (allocate) และคืน (deallocate) ให้ระบบ
- Rust ไม่ใช้ Garbage Collector และไม่ให้เขียน `free()` เอง แต่ผูกการคืนหน่วยความจำเข้ากับ Ownership + Scope โดยตรง
- **ช่วยแก้ปัญหา** runtime overhead ของภาษาที่มี GC และปัญหาความผิดพลาดจากการจัดการเองแบบ C/C++
- **compiler แทรกการเรียก `drop()` ให้อัตโนมัติตอน compile time** ทำให้ปลอดภัยโดยไม่มี cost ตอน runtime

```
fn main() {  
    let s = String::from("hello"); // ขอหน่วยความจำบน heap  
    println!("{}", s);  
} // ออกจาก scope -> Rust คืนหน่วยความจำอัตโนมัติ ไม่ต้องเขียน free()
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
hello
```

Detailed Code + Live Demo

```
fn main() {  
    let s1 = String::from("hello");  
    let s2 = s1;  
    println!("{}", s2);  
  
}
```

```
fn main() {  
    let s: String = String::from("Hello, world!");  
    print_string(s.clone());  
    println!("{}", s);  
}  
  
fn print_string(s: String) {  
    println!("{}", s);  
}
```


Demo Checklist (ทำไว้เป็น note ของ slide แทน)

- [ขั้นตอน Demo ที่ 1]
ลอง print s2 ให้ดู อธิบายว่า ownership อยู่ที่ s1
และ let s2 = s1 ทำให้ ownership ย้ายไปที่ s2
- [ขั้นตอน Demo ที่ 2]
เปลี่ยน println!("{}", s2) เป็น println!("{}", s1)
- [ขั้นตอน Demo ที่ 3]
อธิบาย error ว่าทำไม s1 ใช้ไม่ได้
- [สิ่งที่ต้องระวังระหว่าง Demo]
เขียนโค้ดผิด

Demo Checklist (ทำไว้เป็น note ของ slide แทน)

- [ขั้นตอน Demo ที่ 1]
อธิบายว่า function `print_string` ทำอะไร
- [ขั้นตอน Demo ที่ 2]
อธิบายว่าตอนส่ง `s` ไปเป็น parameter ownership ถูกย้าย
- [ขั้นตอน Demo ที่ 3]
อธิบายว่าใช้ `clone` ทำไม
- [สิ่งที่ต้องระวังระหว่าง Demo]
วงเล็บ

PPL Perspective — Ownership

Syntax

```
let x = String :: from("hello");
```

```
let y = x;
```

- `let` คือ การสร้าง Variable Binding
- `=` คือ การกำหนด / Binding
- `String` เป็นค่าที่มี Resource ที่ต้องจัดการ

Semantics

```
let x = String :: from("hello");
```

`x` เป็นเจ้าของค่า ต่อมา

```
let y = x;
```

คือคำสั่ง Ownership ย้ายจาก $x \rightarrow y$

ตอนนี้ `x` ไม่มีค่าไม่สามารถใช้ `x` ต่อได้

- Move Semantics by Default: การกำหนดค่าหรือส่งเข้าฟังก์ชัน คือการโอนย้ายสิทธิ์ ตัวแปรต้นทางจะไร้ผล (invalidated) ทันที
- Compile-Time Rejection: หากพยายามใช้ตัวแปรที่ถูกย้ายสิทธิ์ไปแล้ว Compiler จะปฏิเสธไม่ให้นำไปรวม

PPL Perspective — Ownership

Type System

- Affine Type System: ระบบ Type ควบคุมไม่ให้ Resource ถูกใช้งานแบบละเมิดสิทธิ์ (ใช้งานได้มากที่สุด 1 ครั้ง)
- Copy vs Move Differentiation: แยกแยะพฤติกรรมตามชนิดผ่าน Type (เช่น String ใช้ Move, i32 ใช้ Copy)
- Static Analysis: ตรวจสอบกฎ Ownership และการเข้าถึงข้อมูลทั้งหมดตั้งแต่ช่วง Compile Time

Memory / Resource

- Deterministic Destruction: เมื่อ Owner ออกจาก Scope Rust จะดำเนินการทำลาย Value ตามกฎของภาษา โดยสำหรับ Type ที่มี Drop จะมีการเรียกกระบวนการ drop เพื่อจัดการ Resource ที่เกี่ยวข้องโดยอัตโนมัติคืนหน่วยความจำทันทีที่ตัวแปรหลุดจากขอบเขต (Out of Scope)
- ผลที่ได้คือการจัดการ Resource มีลักษณะที่คาดการณ์ได้ (Deterministic) และไม่จำเป็นต้องใช้ Garbage Collector เพื่อค้นหา Object ที่ไม่มีการใช้งานแล้ว

PPL Perspective — Ownership

Abstraction

- Ownership สามารถมองได้ว่าเป็น Abstraction สำหรับ Resource Management ที่ทำให้ Programmer สามารถควบคุมอายุการใช้งานและความรับผิดชอบต่อ Resource ผ่านกฎของภาษา แทนที่จะต้องจัดการ Pointer และการคืน Resource ด้วยตนเองในระดับต่ำ
- แนวคิดดังกล่าวเชื่อมโยงกับแนวคิดพื้นฐานของ PPL หลายประการ ได้แก่
- Scope & Binding: Scope กำหนดขอบเขตการใช้งานของ Binding และมีความสัมพันธ์กับช่วงเวลาที่ตัวแปรสามารถเป็น Owner ของ Value ได้ ดังนั้น Binding ใน Rust จึงมีความหมายมากกว่าการจับคู่ชื่อกับค่า แต่ยังสัมพันธ์กับสถานะและความรับผิดชอบต่อ Resource
- Aliasing & Mutability: การมีหลายส่วนของโปรแกรมเข้าถึงข้อมูลเดียวกัน (Aliasing) พร้อมกับการแก้ไขข้อมูล (Mutability) สามารถทำให้เกิดปัญหาด้าน Memory Safety ได้ Rust จึงใช้ Ownership และ Borrowing Rules เป็นส่วนหนึ่งของกลไกในการควบคุมการเข้าถึงข้อมูลดังกล่าว โดยรายละเอียดของ References และ Borrowing จะกล่าวถึงในหัวข้อที่เกี่ยวข้องโดยเฉพาะ

Rust vs. Other Language

- เปรียบเทียบตาม Memory Management แบ่งเป็น 3 กลุ่ม คือ
- **Manual Memory Management** คือ Programmer มีบทบาทในการจัดการ lifetime/resource เอง เช่น malloc/free, new/delete หรือใช้ RAII/smart pointers ใน C++ เช่น C, C++
- **Automatic Memory Management** คือ Runtime / Garbage Collector หรือ GC ช่วยจัดการ memory lifetime เช่น Java, Python, C#, Go
- **Ownership-based Memory Management** คือ Ownership + Borrow Checking ตรวจสอบกฎสำคัญตอน Compile Time โดยไม่ต้องใช้ GC สำหรับ memory management ปกติ เช่น Rust

Rust vs. Other Language

Aspect	C	C++	Java	Python	C#	Go	Rust
Memory Model	Manual	Manual + RAII	GC	GC / ref counting + GC	GC	GC	Ownership
Memory Deallocation	Programmer	RAII / programmer / smart pointers	GC	Automatic	GC	GC	Scope / Drop
Garbage Collector	No	No	Yes	Yes*	Yes	Yes	No
Ownership Checking	No built-in ownership system	No Rust-like ownership system	No	No	No	No	Yes
Compile-time Memory Safety	Limited	Depends on usage/features	Partial	Partial	Partial	Stronger runtime/type safety	Strong
Main Trade-off	Control vs safety	Control + abstractions	Runtime GC	Runtime management	Runtime GC	Runtime GC	Compile-time checking

Rust vs. C

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char *message = malloc(20);

    if (message != NULL) {
        snprintf(message, 20, "Hello C");
        printf("%s\n", message);

        free(message);
    }

    return 0;
}
```

1. ภาษา C: Manual Memory Management (การจัดการด้วยตัวเอง)

- การทำงาน: malloc(20) จะจองพื้นที่บน Heap ขนาด 20 ไบต์ และต้องใช้ if (message != NULL) เพื่อเช็คว่าจองสำเร็จไหม จากนั้นเมื่อใช้งานเสร็จต้องเรียก free(message) เพื่อคืนรันไทม์เสมอ
- ข้อดี: ทำงานได้เร็วที่สุด ไม่กินทรัพยากรเครื่องโดยไม่จำเป็น ควบคุมฮาร์ดแวร์ได้ตรงตัว
- ข้อจำกัด: หากลืมเขียน free() จะเกิดปัญหา Memory Leak หรือหากเผลอไปสั่ง d = null (เหมือน Java) โดยไม่ free พื้นที่ตรงนั้นจะฝังอยู่ในแรมไปตลอดจนกว่าจะปิดโปรแกรม

Rust vs. Java

```
class Data {  
    String message;  
  
    public Data(String message) {  
        this.message = message;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        Data d = new Data("Hello Java");  
  
        System.out.println(d.message);  
  
        d = null;  
    }  
}
```

2. ภาษา Java: Garbage Collection

- การทำงาน: เมื่อโปรแกรมเมอร์ใช้คำสั่ง new ตัว Java Virtual Machine (JVM) จะจองพื้นที่บน Heap ให้ และเมื่อเราสั่ง d = null; ตัวแปร d จะไม่ได้ชี้ไปที่วัตถุนั้นแล้ว หลังจากนั้นตัว Garbage Collector (GC) จะแอบเข้ามาเคลียร์หน่วยความจำทิ้งให้เองในภายหลัง
- ข้อดี: เขียนง่าย ปลอดภัย โอกาสเกิด Memory Leak น้อยลงมาก โปรแกรมไม่ค่อยพังจากเรื่องแรม
- ข้อจำกัด: การทำงานของ GC จะกินทรัพยากรเครื่อง และบางครั้งอาจทำให้โปรแกรมเกิดอาการ "กระตุก" (Stop-the-world) เป็นช่วงสั้น ๆ เพื่อเคลียร์ขยะ

Rust

```
fn main() {  
    let message = String::from("Hello Rust");  
    println!("{}", message);  
}
```

3. ภาษา Rust: Ownership & Automatic Dropping (ระบบความเป็นเจ้าของ)

- การทำงาน: Rust ใช้ระบบ Ownership (ความเป็นเจ้าของ) เมื่อตัวแปร message ถูกสร้างขึ้น มันจะเป็นเจ้าของหน่วยความจำก้อนนั้นทันที และเมื่อรันมาถึงปีกกาปิด } ของฟังก์ชัน ตัวแปร message จะ "หลุดออกจาก Scope" ระบบของ Rust จะใส่คำสั่งคืนหน่วยความจำ (drop) ให้โดยอัตโนมัติทันทีที่ตอน Compile
- ข้อดี: ปลอดภัยเทียบเท่า Java (หรือมากกว่า) และทำงานได้รวดเร็วเท่าภาษา C เพราะไม่ต้องมีระบบ GC คอยรันเบื้องหลังตอนโปรแกรมทำงาน
- ข้อจำกัด: โปรแกรมเมอร์ต้องเรียนรู้และปฏิบัติตามกฎ Ownership ที่เข้มงวดมาก ไม่งั้นจะ Compile โปรแกรมไม่ผ่าน

Rust Ownership ในมุมมองเชิง Programming Language

Rust Ownership เป็นส่วนสำคัญของการออกแบบภาษาเพื่อสร้าง Memory Safety ผ่านการตรวจสอบในช่วง Compile Time โดยไม่จำเป็นต้องใช้ Garbage Collector เป็นกลไกหลักในการจัดการ Memory แนวทางนี้ช่วยให้ Rust สามารถจัดการ Resource ได้อย่างเป็นระบบและมีค่าใช้จ่ายขณะ Runtime ที่คาดการณ์ได้ ขณะเดียวกันยังสามารถป้องกัน Memory-Safety Bugs หลายประเภท เช่น การใช้ข้อมูลหลังจาก Ownership ถูกย้ายไปแล้ว และปัญหาบางประเภทที่เกี่ยวข้องกับการเข้าถึง Memory อย่างไม่ปลอดภัย ดังนั้น Ownership จึงเป็นตัวอย่างสำคัญของการออกแบบ Programming Language ที่นำกฎด้าน Resource Management และ Safety เข้ามาเป็นส่วนหนึ่งของภาษาและ Compiler แทนที่จะพึ่งพาการตรวจสอบโดย Programmer หรือ Garbage Collector เพียงอย่างเดียว

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

1. การพยายามใช้ค่าที่ถูกย้าย (Move) ไปแล้ว

✗ Incorrect

```
fn print(message: String) {
    println!("{}", message);
}

fn main() {
    let message = String::from("It's not okay!");
    print(message);
    println!("{}", message);
}
```

✓ Correct

```
fn print(message: String) {
    println!("{}", message);
}

fn main() {
    let message = String::from("It's not okay!");
    print(message.clone());
    println!("{}", message);
}
```

ทำไมถึงผิด? (Why?)

my_name ถูกส่งไปยังฟังก์ชัน print_name() สิทธิ์ความเป็นเจ้าของ (ownership) ของ String จะถูกย้ายไปยังฟังก์ชันนั้น ดังนั้น my_name จึงไม่สามารถนำมาใช้งานต่อใน main() ได้

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

2. เข้าใจผิดว่าการ assign คือการ copy ทั้งที่จริงๆ คือการ move

✗ Incorrect

```
fn main() {  
    let s1 = String::from("move not copy");  
    let s2 = s1;          // ความเป็นเจ้าของถูกย้ายจาก s1 ไป s2  
    println!("{}", s1);   // ERROR: s1 ใช้งานไม่ได้แล้ว  
}
```

✓ Correct

```
fn main() {  
    let s1 = String::from("move not copy");  
    let s2 = s1.clone();  
    println!("{}", s1);  
}
```

ทำไมถึงผิด? (Why?)

มาจากภาษาอย่าง Python, Java หรือ JS การเขียน `let s2 = s1`; อาจดูเหมือนแค่สร้างตัวแปรตัวที่สองที่ชี้ไปยังข้อมูลเดียวกัน แล้วใช้ได้ทั้งสองชื่อ แต่ใน Rust สำหรับ type ที่ไม่ใช่ Copy นี่คือการ move ไม่ใช่การ copy `s1` จะใช้งานไม่ได้ทันทีที่ `s2` ถูกสร้างขึ้น

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

3. Move บางส่วนออก struct (Partial move)

✗ Incorrect

```
fn main() {
    let user = User { name: String::from("Alice"),
age: 30 };

    let name = user.name;
    println!("{}", user.name);
    println!("{}", user.age);
}
```

✓ Correct

```
fn main() {
    let s1 = String::from("move not copy");
    let s2 = s1.clone();
    println!("{}", s1);
}
```

ทำไมถึงผิด? (Why?)

การย้าย field เดียวออกจาก struct จะทำให้ struct นั้น "ใช้งานไม่ได้บางส่วน" จะใช้ struct ทั้งก้อน (หรือ field ที่ถูกย้ายไปนั้น) อีกไม่ได้ ถึงแม้ field อื่นๆ จะยังใช้งานได้ปกติก็ตาม จุดนี้มักทำให้คนงงตอนแรกทีเดียว เพราะ error message อาจดูสับสน struct ยัง "มีอยู่" แต่บาง field ในนั้นใช้ไม่ได้แล้ว

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

4. Move ค่าเข้าไปใน loop แล้วพยายามใช้ซ้ำ

✗ Incorrect

```
fn greet(name: String) {
    println!("Hello, {}", name);
}

fn main() {
    let name = String::from("Alice");

    for i in 0..3 {
        greet(name);
    }
}
```

✓ Correct

```
fn greet(name: String) {
    println!("Hello, {}", name);
}

fn main() {
    let name = String::from("Alice");

    for i in 0..3 {
        greet(name.clone());
    }
}
```

ทำไมถึงผิด? (Why?)

การย้าย field เดียวออกจาก struct จะทำให้ struct นั้น "ใช้งานไม่ได้บางส่วน" จะใช้ struct ทั้งก้อน (หรือ field ที่ถูกย้ายไปนั้น) อีกไม่ได้ ถึงแม้ field อื่นๆ จะยังใช้งานได้ปกติก็ตาม จุดนี้มักทำให้คนงงตอนแรกทีเดียว เพราะ error message อาจดูสับสน struct ยัง "มีอยู่" แต่บาง field ในนั้นใช้ไม่ได้แล้ว

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

5. Anti-Pattern: "Clone ทุกอย่าง"

✗ Incorrect

```
fn main() {
    let user = User {
        name: String::from("Alice"),
        email: String::from("alice@su.ac.th"),
        bio: String::from("no bio added"),
    };

    print_name(user.name.clone()); // clone() ทั้งที่แค่จะ print เฉยๆ
    println!("{}", user.name);    // ต้องใช้ user.name ต่อ เลย clone
    ไปก่อน
}
```

✓ Correct

```
fn main() {
    let user = User {
        name: String::from("Alice"),
        email: String::from("alice@su.ac.th"),
        bio: String::from("no bio added"),
    };
    print_name(&user.name);        // แค่อ้าง ไม่ต้อง clone
    println!("{}", user.name);
}
```

ทำไมถึงผิด? (Why?)

แม้จะไม่ใช่วิธีข้อผิดพลาดระดับคอมไพเลอร์ แต่นี่คือข้อผิดพลาดทางพฤติกรรม เมื่อ Borrow Checker แจ้งเตือนข้อผิดพลาด ผู้เริ่มต้นมักจะใส่ `.clone()` ไว้ในทุกตัวแปร เพียงเพื่อบังคับให้โค้ดสามารถคอมไพล์ผ่าน

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem): Clone or Lose It

จะเจอ compile error หน้าที่ของคุณคือ แก้โค้ด ให้ขั้นตอนที่ 3 compile ผ่าน โดยไม่เปลี่ยนพฤติกรรมของ describe (ยังต้องรับความเป็นเจ้าของ String เหมือนเดิม)



Hint

describe รับ item: String แบบ by value ดังนั้นการเรียก describe(item) จะย้ายความเป็นเจ้าของออกไปจาก item ใน main ต้องหาวิธีที่ทำให้ item ยังใช้งานได้หลังจากนั้น โดยไม่เปลี่ยน signature ของ describe มีคำสั่งอะไรที่ช่วยให้ส่ง *สำเนา* ไปแทนตัวจริงได้บ้าง?

```

1  fn describe(item: String) -> String {
2      format!("Item: {}", item)
3  }
4
5  fn main() {
6      let item = String::from("Book");
7
8      let result = describe(item);
9      println!("{}", result);
10
11     println!("{}", item);
12 }

```

Solution

line 8: let result = describe(item.clone());

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem): The Half-Moved Book

คุณได้รับ struct `Book` ที่มี field เป็น `String` สองตัว และมี loop ที่ต้องการสร้าง label สำหรับแต่ละเล่ม แล้ว print ข้อมูลเต็มของแต่ละเล่มที่หลัง



Hint

book.title ถูกย้ายเข้าไปใน make_label(book.title) หลังจากบรรทัดนั้น book.title ยังใช้งานได้อยู่ไหม? แล้ว field อื่นของ book (เช่น book.author) ยังใช้ได้ปกติหรือเปล่า? นี่เป็นปัญหาแบบเดียวกับการเข้าถึง field ของ struct หลังจากบางส่วนถูกย้ายไปแล้ว ลองคิดว่า field ไหนที่ต้องรอดจนถึงหลังจากเรียกฟังก์ชันนั้น แล้วจะทำยังไงให้มันรอด

```

1 struct Book {
2     title: String,
3     author: String,
4 }
5
6 fn make_label(title: String) -> String {
7     format!("{}", title)
8 }
9
10 fn main() {
11     let books = vec![
12         Book { title: String::from("Rust programming"), author: String::from("Graydon Hoare") },
13         Book { title: String::from("C programming"), author: String::from("Dennis Ritchie") },
14     ];
15
16     for book in books {
17         let label = make_label(book.title);
18         println!("{}", label);
19         println!("by {} - full title: {}", book.author, book.title);
20     }
21 }
```

Solution

1. line 17: `let label = make_label(book.title.clone());`
2. Have make_label return ownership back
`fn make_label: (title: String) -> (String, String) { let label = format!("{}", title); (label, title) }`
3. Reorder use the field before moving it:

```

for book in books {
    println!("by {} - full title: {}", book.author, book.title);
    let label = make_label(book.title);
    println!("{}", label);
}
```

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem): The Half-Moved Book

จากคำถามก่อนหน้านี้ (วิธีที่ 2): แค่เปลี่ยน function เราจะรันโค้ดของเราได้ไหม ?

```

1 struct Book {
2     title: String,
3     author: String,
4 }
5 // ****
6 fn make_label(title: String) -> (String, String) {
7     let label = format!("{}", title);
8     (label, title)
9 }
10 // ****
11 fn main() {
12     let books = vec![
13         Book { title: String::from("Rust programming"), author: String::from("Graydon Hoare") },
14         Book { title: String::from("C programming"), author: String::from("Dennis Ritchie") },
15     ];
16
17     for book in books {
18         let label = make_label(book.title);
19         println!("{}", label);
20         println!("by {} - full title: {}", book.author, book.title);
21     }
22 }
```

```

1 struct Book {
2     title: String,
3     author: String,
4 }
5 fn make_label(title: String) -> (String, String) {
6     let label = format!("{}", title);
7     (label, title)
8 }
9 fn main() {
10     let books = vec![
11         Book { title: String::from("Rust programming"), author: String::from("Graydon Hoare") },
12         Book { title: String::from("C programming"), author: String::from("Dennis Ritchie") },
13     ];
14
15     for book in books {
16         // ****
17         let (label, title) = make_label(book.title);
18         book.title = title;
19         // ****
20         println!("{}", label);
21         println!("by {} - full title: {}", book.author, book.title);
22     }
23 }
```

```

1 struct Book {
2     title: String,
3     author: String,
4 }
5 fn make_label(title: String) -> (String, String) {
6     let label = format!("{}", title);
7     (label, title)
8 }
9 fn main() {
10     let books = vec![
11         Book { title: String::from("Rust programming"), author: String::from("Graydon Hoare") },
12         Book { title: String::from("C programming"), author: String::from("Dennis Ritchie") },
13     ];
14     //****
15     for mut book in books {
16         //****
17         let (label, title) = make_label(book.title);
18         book.title = title;
19         println!("{}", label);
20         println!("by {} - full title: {}", book.author, book.title);
21     }
22 }
```

Key Takeaways — สรุปสิ่งที่ควรรจำ

- 1 ค่าแต่ละค่าในภาษา Rust จะมีตัวแปรที่เป็นเจ้าของ (Owner) เพียงตัวเดียวเท่านั้น
- 2 ในขณะใดขณะหนึ่ง สามารถมีเจ้าของ (Owner) ได้เพียงหนึ่งเดียวเท่านั้น
- 3 เมื่อตัวแปรที่เป็นเจ้าของหลุดออกจากขอบเขตการทำงาน (Out of scope) ข้อมูลค่านั้นจะถูกลบทิ้ง (Dropped) โดยอัตโนมัติ
- 4 Ownership เป็นตัวอย่างสำคัญของการออกแบบ Programming Language ที่นำกฎด้าน Resource Management และ Safety

References — เอกสารอ้างอิง

- The Rust Programming Language: <https://doc.rust-lang.org/book/>
- Rust by Example : <https://doc.rust-lang.org/rust-by-example/>
- W3School: <https://www.w3schools.com/rust/>
- The Rust Programming Language Book: https://www.scs.stanford.edu/~zyedidia/docs/rust/rust_book.pdf page 82-96

GitHub Repository: <https://github.com/sotheasokea/pl-term-project-rust-ownership>

Questions & Answers

ក្រុម 11 — Ownership